

Database Management System Lab Manual

Guided by: **Prof. Hitesh Rahangdale**

Student: **Himanshu Jadhav**

Branch: **AI & Data Science** | Second Year Engineering

Table of Contents

No.	Practical Title
1	Database Setup and Creation
2	Views and Indexing
3	SQL Queries with Filters and Sorting
4	Multi-Table Joins
5	PL/SQL — Stored Procedures and Functions
6	Triggers and Cursors
7	NoSQL — MongoDB Aggregation and Indexing
8	Case Study — Social Media Analysis using NoSQL
9	MySQL Database Backup and Recovery
10	Mini Project — Guidelines and Topics
—	Reference Implementation — Universe Database System

Practical 1: Database Setup and Creation

1. Aim

To design and implement a relational database schema for a Student Course Management System, including table creation, relationship enforcement using primary and foreign keys, and data retrieval using SQL queries.

2. Theory

a) Concept Introduction

A relational database organizes data into structured tables (relations) connected through keys. Primary Keys (PK) uniquely identify each row, while Foreign Keys (FK) establish referential integrity between related tables.

b) Key Concepts

- **Entity:** A real-world object represented as a table (e.g., Student, Course).
- **Attribute:** A property of an entity, represented as a column.
- **Primary Key:** A unique, non-null identifier for each record in a table.
- **Foreign Key:** A column that references the primary key of another table.
- **One-to-Many Relationship:** One record in a parent table links to multiple records in a child table.
- **Many-to-Many Relationship:** Resolved using a junction table (e.g., Enrollment links Student and Course).

c) Syntax

```
CREATE TABLE TableName ( column1 DATATYPE PRIMARY KEY,  
column2 DATATYPE NOT NULL, column3 DATATYPE, FOREIGN  
KEY (column3) REFERENCES OtherTable(column)  
);
```

d) Example Explanation

The Student and Course tables are linked through the Enrollment junction table. This resolves the many-to-many relationship and stores additional data such as EnrollmentDate. The Instructor table is linked to Course in a one-to-many relationship.

e) Advantages / Applications

- Eliminates data redundancy using normalization.
- Ensures data integrity via constraints and foreign keys.
- Forms the structural foundation of enterprise applications (ERP, CRM).
- Enables efficient data retrieval through relational joins.
- Supports scalability for large, multi-user systems.

3. ER Diagram

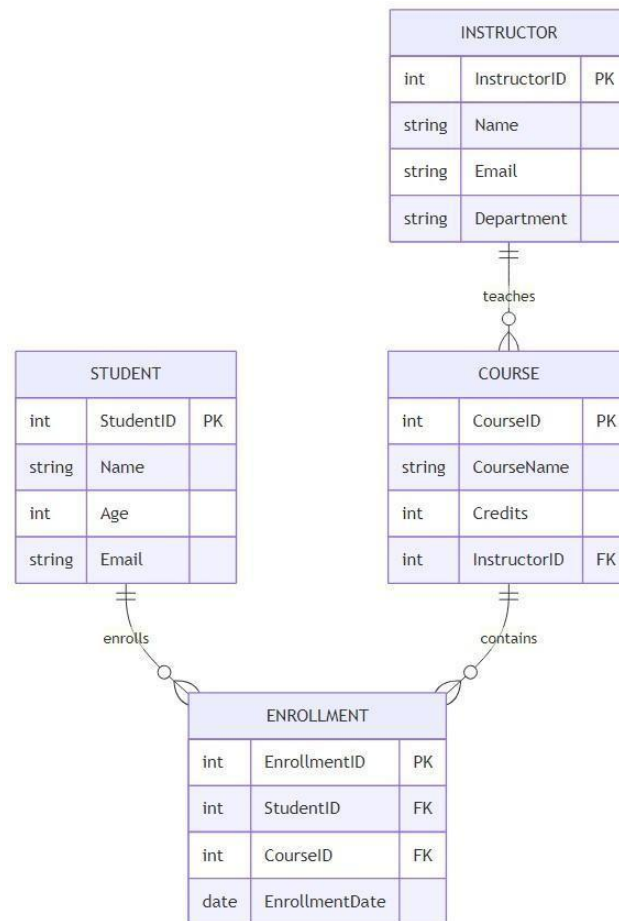


Figure: ER Diagram representing the Student Course Management System

4. Result

All SQL queries for table creation, data insertion, and relational joins were executed successfully in the MySQL environment.

5. Conclusion

This practical demonstrated the process of designing a normalized relational schema using entities, attributes, and relationship constraints. It established a clear understanding of how primary and foreign keys maintain referential integrity and how junction tables resolve many-to-many relationships in real-world database design.

Practical 2: Views and Indexing

1. Aim

To optimize a Student Course Management System by creating database views for simplified data reporting and implementing indexes to enhance query performance.

2. Theory

a) Concept Introduction

A View is a virtual table created from a SELECT query on one or more base tables. An Index is a performance optimization structure that speeds up data retrieval by allowing the database to locate rows without scanning the entire table.

b) Key Concepts

- **Simple View:** Derived from a single table; supports INSERT, UPDATE, DELETE.
 - **Complex View:** Derived from joins across multiple tables; generally read-only.
 - **B-Tree Index:** Default index type; effective for equality and range queries.
 - **Unique Index:** Ensures no duplicate values exist in the indexed column.
 - **Read Trade-off:** Indexes speed up reads but add overhead on write operations (INSERT/UPDATE/DELETE).
 - **EXPLAIN ANALYZE:** A MySQL/PostgreSQL command used to inspect the query execution plan.
- #### c) Syntax

```
-- Creating a View
CREATE VIEW ViewName AS SELECT col1, col2 FROM Table1 JOIN Table2 ON ...;

-- Creating an Index CREATE INDEX IndexName ON TableName (ColumnName);

-- Analyzing query performance
EXPLAIN SELECT * FROM ViewName WHERE condition;
```

d) Example Explanation

A view StudentCourseReport is created by joining the Student, Enrollment, and Course tables to display student names, enrolled courses, and grades in a single, simplified interface. An index on Enrollment.StudentID ensures this join is processed efficiently.

e) Advantages / Applications

- Views enforce security by restricting direct table access to sensitive columns.
- Simplifies complex reporting queries into reusable, clean interfaces.
- Indexes reduce query execution time significantly on large datasets.
- Used in business intelligence dashboards and reporting engines.
- Enables query abstraction without exposing underlying schema structure.

3. ER Diagram

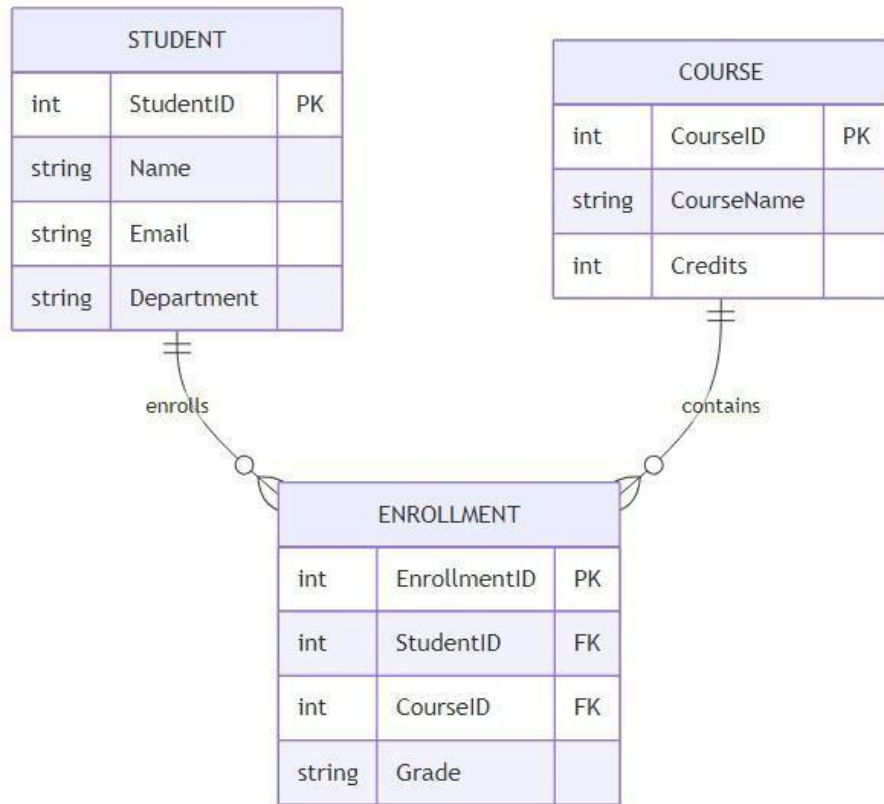


Figure: ER Diagram representing the schema used for Views and Indexing

4. Result

Views were created successfully to abstract reporting data, and indexes were applied on key columns.

Query execution times were observed and compared using EXPLAIN ANALYZE.

5. Conclusion

This practical illustrated how views provide a logical, secure abstraction layer over raw table data and how strategic indexing significantly improves database query performance. The trade-off between read speed and write overhead must be carefully managed in production systems.

Practical 3: SQL Queries with Filters and Sorting

1. Aim

To retrieve meaningful data from a relational database using advanced SQL filtering, sorting, aggregation, and nested subqueries on a Student Course Management System.

2. Theory

a) Concept Introduction

SQL provides powerful clauses to filter, organize, and summarize relational data. The WHERE, ORDER BY, GROUP BY, and subquery constructs enable precise, analytical data retrieval for reporting and decision-making.

b) Key Concepts

- **WHERE Clause:** Filters rows based on one or more conditional expressions.
- **ORDER BY Clause:** Sorts the result set in ascending (ASC) or descending (DESC) order.
- **GROUP BY Clause:** Groups rows with identical column values for aggregation.
- **Aggregate Functions:** COUNT(), SUM(), AVG(), MIN(), MAX() operate on grouped data.
- **HAVING Clause:** Filters grouped results after aggregation (analogous to WHERE for groups).
- **Nested Subquery:** A query embedded inside another query's WHERE, FROM, or SELECT clause.

c) Syntax

```
-- Filtering and Sorting
SELECT col1, col2
FROM Table
WHERE condition
ORDER BY col1 ASC;

-- Aggregation with Grouping
SELECT col1, COUNT(*) AS
total FROM Table
GROUP BY col1
HAVING COUNT(*) > 1;

-- Nested Subquery
SELECT Name FROM Student
WHERE StudentID IN ( SELECT StudentID FROM Enrollment WHERE
CourseID = 101
);
```

d) Example Explanation

To find all students above 20 years of age enrolled in a specific course, a WHERE clause filters by age and a nested subquery retrieves the relevant StudentID values from the Enrollment table. GROUP BY is used to count the number of students per course.

e) Advantages / Applications

- WHERE and ORDER BY improve user experience in search-driven applications.
- Aggregation functions power analytical dashboards and business reports.
- Nested subqueries enable complex, multi-level data filtering without temporary tables.
- HAVING supports post-aggregation filtering critical in data warehousing.
- These techniques are foundational for backend API query engineering.

3. ER Diagram

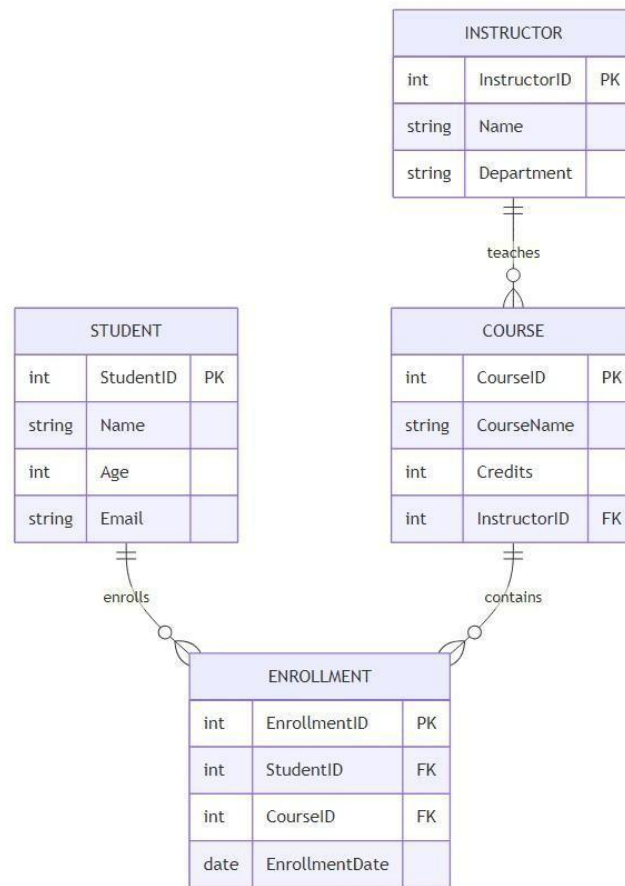


Figure: ER Diagram representing the schema used for Filtering and Sorting queries

4. Result

SQL queries involving WHERE, ORDER BY, GROUP BY, HAVING, and nested subqueries were executed successfully, producing accurate and structured result sets.

5. Conclusion

This practical reinforced the importance of precise data retrieval in relational databases. Advanced filtering and aggregation techniques are indispensable in real-world applications including analytics platforms, e-commerce dashboards, and academic reporting systems.

Practical 4: Multi-Table Joins

1. Aim

To implement and analyze various SQL JOIN operations — INNER, LEFT, RIGHT, and FULL OUTER JOIN — in a Customer Order Management System to retrieve combined relational data.

2. Theory

a) Concept Introduction

SQL JOINS combine rows from two or more tables based on a related column, enabling retrieval of associated data distributed across a normalized schema. Different join types define how unmatched rows are handled.

b) Key Concepts / Types

- **INNER JOIN:** Returns only rows where a matching record exists in both tables.
- **LEFT JOIN (LEFT OUTER JOIN):** Returns all rows from the left table; unmatched right rows appear as NULL.
- **RIGHT JOIN (RIGHT OUTER JOIN):** Returns all rows from the right table; unmatched left rows appear as NULL.
- **FULL OUTER JOIN:** Returns all rows from both tables. In MySQL, simulated using UNION of LEFT and RIGHT JOINS.
- **Referential Integrity:** Enforced via FOREIGN KEY constraints, ensuring join validity.
- **EXPLAIN:** Analyzes the query execution plan to detect and optimize inefficient joins.

c) Syntax

```
-- INNER JOIN SELECT a.col, b.col FROM TableA a INNER JOIN TableB b ON a.id = b.id;

-- LEFT JOIN SELECT a.col, b.col FROM TableA a LEFT JOIN TableB b ON a.id = b.id;

-- FULL OUTER JOIN (MySQL Simulation) SELECT * FROM TableA LEFT JOIN TableB
ON TableA.id = TableB.id
UNION
SELECT * FROM TableA RIGHT JOIN TableB ON TableA.id = TableB.id;
```

d) Example Explanation

In a Customer Order system, an INNER JOIN retrieves only customers who have placed orders. A LEFT JOIN reveals customers with no orders (NULL in the orders column). A RIGHT JOIN identifies orphaned orders not linked to any customer.

e) Advantages / Applications

- INNER JOINS are standard for data retrieval in normalized schemas.
- LEFT JOINS identify missing or incomplete data relationships.
- FULL OUTER JOINS are critical in data migration and ETL pipelines.
- Used extensively in e-commerce, CRM, and billing systems.
- Proper join indexing can reduce query time by orders of magnitude.

3. ER Diagram

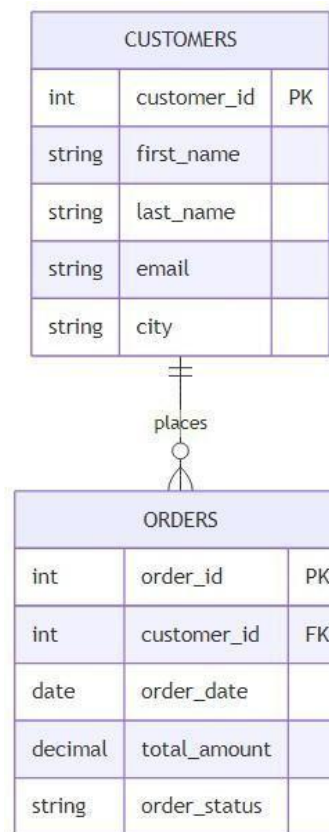


Figure: ER Diagram representing the Customer Order Management System

4. Result

All four join types — INNER, LEFT, RIGHT, and FULL OUTER JOIN — were executed successfully. Query performance was validated using EXPLAIN, and index on orders.customer_id confirmed optimized join execution.

5. Conclusion

This practical provided a comprehensive understanding of relational join operations and their real-world significance. Selecting the appropriate join type is crucial for correctness and performance in database-driven applications.

Practical 5: PL/SQL — Stored Procedures and Functions

1. Aim

To implement and demonstrate the use of stored procedures and functions in MySQL by building a Sales Management System that automates order processing and revenue calculation at the database level.

2. Theory

a) Concept Introduction

Stored Procedures and Functions are reusable, pre-compiled database objects that encapsulate SQL logic. They centralize business logic within the database, improving modularity, security, and execution performance.

b) Key Concepts

- **Stored Procedure:** A named block of SQL statements that can accept parameters and perform DML operations. Does not return a value directly.
- **Stored Function:** Similar to a procedure but must return a single scalar value. Used in SELECT queries.
- **IN / OUT Parameters:** Procedures accept input (IN) and can return values via OUT parameters.
- **DELIMITER:** Changed to // or \$\$ in MySQL to define multi-statement procedures cleanly.
- **CALL Statement:** Used to invoke a stored procedure.
- **Deterministic Function:** Returns the same result for the same inputs; declared using DETERMINISTIC keyword.

c) Syntax

```
-- Stored Procedure
DELIMITER // CREATE PROCEDURE ProcedureName(IN param1
DATATYPE)
BEGIN -- SQL
Statements
END //
DELIMITER ;

-- Stored Function
DELIMITER // CREATE FUNCTION FunctionName(param1
DATATYPE) RETURNS DATATYPE DETERMINISTIC
BEGIN RETURN
expression;
END //
DELIMITER ;

-- Invoking
CALL ProcedureName(value);
SELECT FunctionName(value);
```

d) Example Explanation

sp_add_order is a stored procedure that inserts a new order record along with its associated items in an atomic block. fn_total_sales is a function that computes total revenue for a given customer by summing quantity × unit_price across their order items.

e) Advantages / Applications

- Reduces application-server round trips by executing logic directly in the database.
- Improves security by restricting direct table access to only stored routines.
- Pre-compilation ensures faster and consistent query execution.

- Widely used in banking, ERP, and inventory management systems.
- Facilitates code reuse and simplifies complex multi-step business operations.

3. ER Diagram

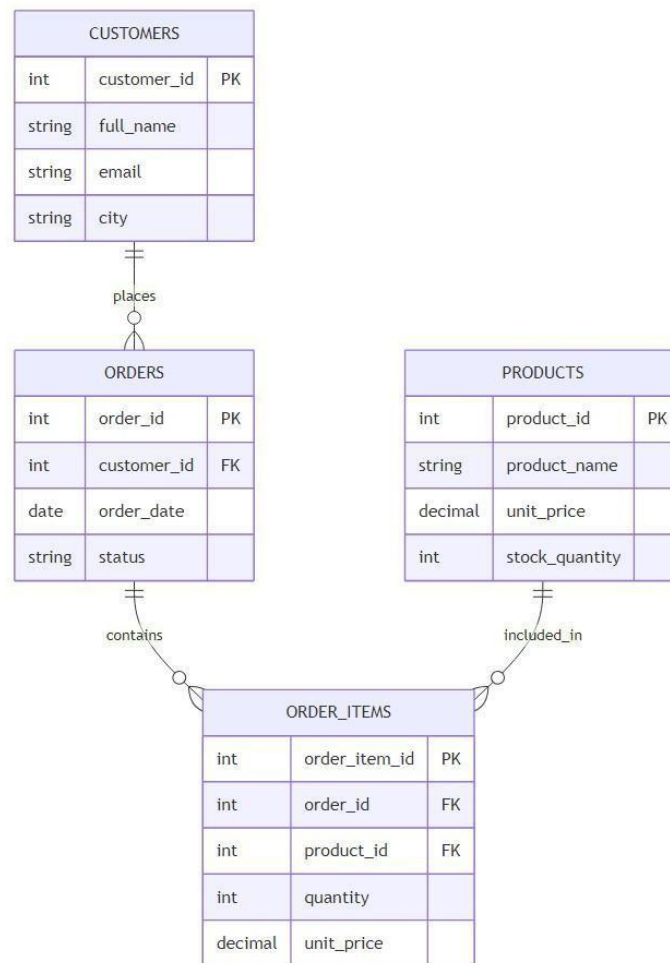


Figure: ER Diagram representing the Sales Management System

4. Result

Stored procedure `sp_add_order` and function `fn_total_sales` were created and executed successfully, demonstrating automated order insertion and revenue aggregation at the database level.

5. Conclusion

This practical underscored the value of database-level programming for automating repetitive logic, enforcing business rules, and improving application performance. Stored routines are essential components of enterprise-grade database architecture.

Practical 6: Triggers and Cursors

1. Aim

To demonstrate the use of AFTER triggers for automated audit logging and cursor-based stored procedures for row-by-row conditional salary updates in an Employee Management System.

2. Theory

a) Concept Introduction

A Trigger is a database object that automatically executes a predefined SQL block in response to an INSERT, UPDATE, or DELETE event on a table. A Cursor allows sequential, row-by-row traversal and manipulation of a query result set.

b) Key Concepts

- **BEFORE Trigger:** Fires before the DML operation; can modify the data being written.
- **AFTER Trigger:** Fires after the DML operation is committed; ideal for audit logging.
- **FOR EACH ROW:** Executes the trigger body once per affected row.
- **OLD / NEW:** References previous and updated row values within a trigger.
- **Explicit Cursor:** Declared and controlled manually using OPEN, FETCH, and CLOSE.
- **FETCH Statement:** Retrieves the next row from the cursor's result set into declared variables.
- **Cursor Loop:** Uses a LOOP or WHILE construct with the cursor to iterate until NOT FOUND.

c) Syntax

```
-- AFTER Trigger
CREATE TRIGGER trigger_name AFTER INSERT ON
employees FOR EACH ROW
BEGIN INSERT INTO employee_audit (employee_id, action_type, changed_at) VALUES
(NEW.employee_id, 'INSERT', NOW());
END;

-- Cursor in a Procedure DECLARE cur CURSOR FOR SELECT employee_id, salary FROM
employees; DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE; OPEN
cur; fetch_loop: LOOP FETCH cur INTO v_id, v_salary; IF done THEN LEAVE fetch_loop;
END IF;
END LOOP;
CLOSE cur;
```

d) Example Explanation

Three AFTER triggers (INSERT, UPDATE, DELETE) on the employees table automatically record old and new salary values, action type, and timestamp into the employee_audit table. A cursor-based procedure iterates all employees in the IT department and applies a 10% salary increment conditionally.

e) Advantages / Applications

- Triggers enforce audit trails without modifying application-layer code.
- Automates data synchronization and maintains derived or historical data.
- Cursors enable complex conditional logic that cannot be expressed in set-based SQL.
- Widely used in financial systems for regulatory compliance and transaction logging.
- Ensures data consistency during complex, multi-step administrative operations.

3. ER Diagram

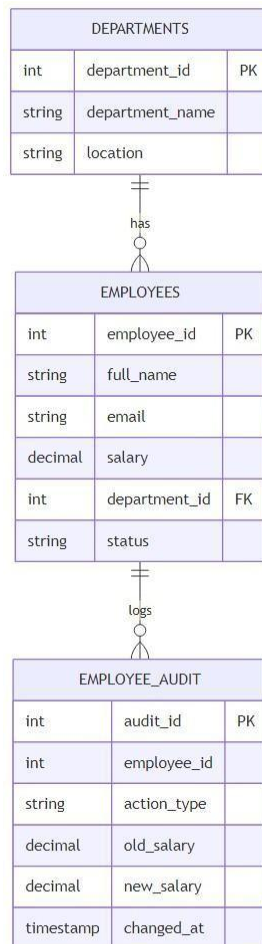


Figure: ER Diagram representing the Employee Management System with Audit Trail

4. Result

AFTER triggers for INSERT, UPDATE, and DELETE were created and tested successfully. The cursor-based salary update procedure correctly applied conditional increments and all changes were reflected in the audit log.

5. Conclusion

This practical demonstrated the power of triggers for passive, automated data management and cursors for procedural database logic. These constructs are critical in enterprise environments where manual oversight of data changes must be both minimized and fully traceable.

Practical 7: NoSQL — MongoDB Aggregation and Indexing

1. Aim

To understand and implement CRUD operations, aggregation pipelines, and indexing strategies in MongoDB for efficient handling of semi-structured, document-oriented data.

2. Theory

a) Concept Introduction

MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like BSON documents within collections. Unlike relational databases, it does not enforce a fixed schema, making it suitable for evolving and heterogeneous data models.

b) Key Concepts

- **Collection:** Equivalent to a table in RDBMS; stores a set of documents.
- **Document:** A JSON-like object (BSON) representing a single record with key-value pairs.
- **Embedding vs. Referencing:** Embedding stores related data in one document (faster reads); Referencing stores IDs linking to other documents.
- **Aggregation Pipeline:** A series of processing stages — \$match, \$group, \$sort, \$project — applied sequentially to transform documents.
- **Index in MongoDB:** Created using createIndex(); improves lookup speed on specified fields.
- **CRUD:** insertOne, find, updateOne, deleteOne — the four fundamental operations. **c) Syntax**

```
// Insert a document
db.users.insertOne({ name: "Alice", age: 25, city: "Mumbai" });

// Aggregation Pipeline db.users.aggregate([ { $match: { age: { $gt: 20 } } }, { $group: {
  _id: "$city", total: { $sum: "$purchase_amount" } } }, { $sort: { total: -1 } }
]);

// Create an Index
db.users.createIndex({ email: 1 });
```

d) Example Explanation

A users collection stores customer documents with embedded address fields. An aggregation pipeline groups users by city and calculates total purchase amounts. An index on email and age ensures rapid query execution when filtering or sorting on these fields.

e) Advantages / Applications

- Flexible schema accommodates rapidly changing data structures without table migrations.
- Horizontal scaling (sharding) enables MongoDB to handle massive data volumes.
- Aggregation pipelines replace complex multi-join SQL queries with readable, staged logic.
- Indexing in MongoDB mirrors SQL indexing but operates on nested document fields as well.
- Widely used in content management, real-time analytics, IoT, and mobile applications.

3. ER Diagram

Note: MongoDB is schema-less; the following represents the logical data model used in this practical.

USERS		
ObjectId	_id	PK
string	name	
int	age	
string	email	
string	city	
decimal	purchase_amount	

Figure: Logical Data Model representing the Users Collection in MongoDB

4. Result

CRUD operations were performed successfully on the users collection. Aggregation pipelines correctly computed group-level statistics, and indexes on age and email demonstrated measurable query performance improvements.

5. Conclusion

This practical highlighted the fundamental differences between relational and document-oriented databases. MongoDB's aggregation framework and indexing capabilities make it a powerful choice for handling large-scale, flexible, and analytical workloads in modern applications.

Practical 8: Case Study — Social Media Analysis using NoSQL

1. Aim

To implement a real-world social media analytics case study using MongoDB, demonstrating data modeling for unstructured content, aggregation pipeline analysis for engagement metrics, and indexing for optimized querying.

2. Theory

a) Concept Introduction

Social media platforms generate massive volumes of semi-structured data — posts, hashtags, comments, and engagement metrics — that are ill-suited for rigid relational schemas. MongoDB's flexible document model and powerful aggregation framework address these challenges effectively. **b) Key Concepts**

- **Embedded Documents:** User details (username, verified status) stored directly inside each post document, avoiding joins and improving read throughput.
- **Arrays in Documents:** Hashtags stored as an array field within a post document, enabling multi-value searches and trend aggregation.
- **\$unwind Stage:** Deconstructs an array field, producing one output document per array element — essential for hashtag frequency analysis.
- **\$group Stage:** Groups unwound documents by a field and accumulates metrics (count, sum).
- **Compound Index:** An index on multiple fields (e.g., {hashtags: 1, created_at: -1}) used to optimize multi-field query patterns.
- **Text Index:** Enables full-text search across post content fields.

c) Syntax

```
// Trending Hashtags Aggregation
db.posts.aggregate([ { $unwind: "$hashtags"
}, { $group: { _id: "$hashtags", count: { $sum: 1 } } }, { $sort: { count: -1 } },
{ $limit: 5 }
]);

// Compound Index
db.posts.createIndex({ "hashtags": 1, "created_at": -1 });
```

d) Example Explanation

Each post document embeds user info and stores hashtags as an array. The pipeline first \$unwinds the hashtag array, then \$groups by hashtag to count occurrences, \$sorts by frequency in descending order, and \$limits the output to the top 5 trending hashtags.

e) Advantages / Applications

- Handles high-velocity, schema-flexible social media data without costly migrations.
- \$unwind enables flat aggregation of nested array data for trend discovery.
- Powers real-time analytics dashboards for platforms like Twitter and Instagram.
- Compound indexes optimize analytics queries with multiple filter conditions.
- Embedding eliminates expensive join lookups for user-centric data access patterns.

3. ER Diagram

Note: The following represents the logical document model for the posts collection.

POSTS		
ObjectId	_id	PK
string	post_text	
array	hashtags	
int	likes	
int	comments	
int	shares	
string	location	
timestamp	created_at	
boolean	is_verified	

USER_INFO	
string	username
string	display_name
boolean	verified

Figure: Logical Data Model representing the Posts Collection for Social Media Analytics

4. Result

CRUD operations, hashtag trend analysis, engagement ranking, and top-user identification were executed successfully. Compound indexes on hashtags and created_at were verified to reduce document scanning in analytical queries.

5. Conclusion

This case study demonstrated how MongoDB's document model and aggregation pipeline are purpose-built for social media analytics workloads. The \$unwind and \$group operators, combined with strategic indexing, form the backbone of scalable, real-time trend analysis systems.

Practical 9: MySQL Database Backup and Recovery

1. Aim

To design a Library Management System database and demonstrate the complete process of performing logical backup using mysqldump and restoring the database to verify data integrity and disaster recovery readiness.

2. Theory

a) Concept Introduction

Database Backup is the process of exporting and preserving a copy of database schema and data to protect against data loss. Recovery is the process of restoring database state from a backup file after failure, corruption, or accidental deletion.

b) Key Concepts

- **Logical Backup:** Exports the database as human-readable SQL statements (CREATE TABLE, INSERT INTO) using mysqldump. Can be restored to any compatible MySQL instance.
- **Physical Backup:** Copies raw database files (data directory). Faster but less portable.
- **mysqldump:** MySQL command-line utility for creating logical backups of databases or specific tables.
- **Point-in-Time Recovery:** Combining full backup with binary logs allows restoring data to any specific moment.
- **Transaction-Consistent Backup:** Using --single-transaction in mysqldump ensures a consistent snapshot without locking tables.
- **Post-Recovery Validation:** Executing join and aggregation queries after restore to confirm referential integrity is preserved.

c) Syntax

```
-- Perform a logical backup
mysqldump -u root -p --single-transaction database_name > backup.sql

-- Restore from backup
mysql -u root -p database_name < backup.sql

-- Verify integrity after restore
SHOW TABLES;
SELECT COUNT(*) FROM table_name;
```

d) Example Explanation

A Library Management System database (authors, books, members, borrow_records) is backed up using mysqldump. The database is then dropped entirely. The .sql backup file is restored using the mysql client. Post-restore queries — including JOIN validations and COUNT checks — confirm that all records and relationships are intact.

e) Advantages / Applications

- Protects against data loss due to hardware failure, cyber-attacks, or human error.
- Logical backups are portable and compatible across MySQL versions.
- --single-transaction ensures backup consistency in live, transactional databases.
- Essential for DBA certification, compliance, and enterprise SLA requirements.
- Backup scheduling integrated with cron enables automated disaster recovery.

3. ER Diagram

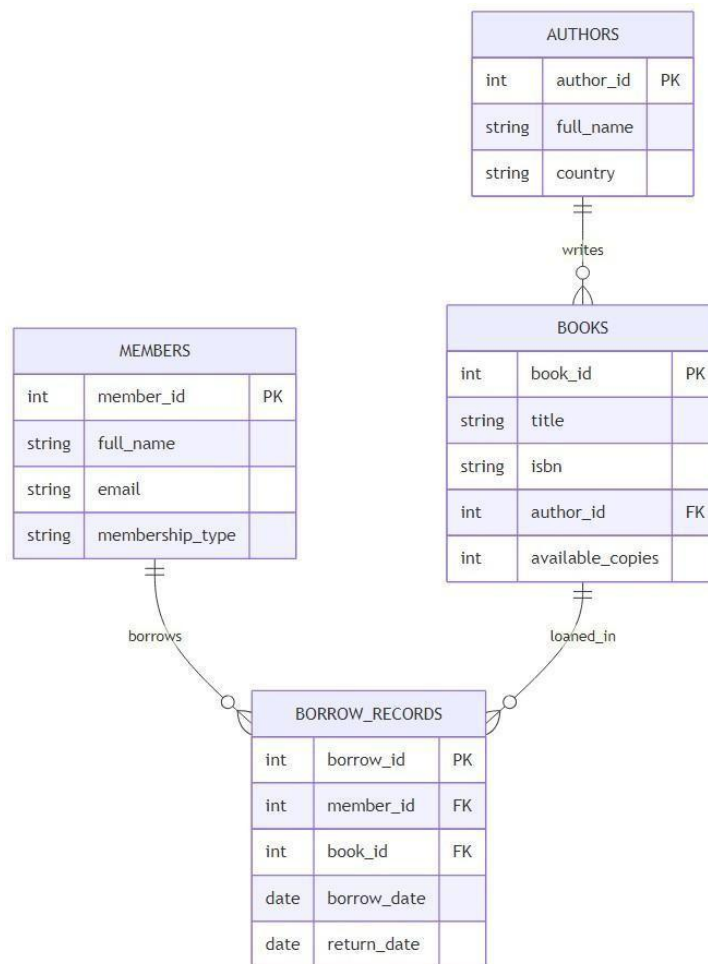


Figure: ER Diagram representing the Library Management System used for Backup and Recovery

4. Result

Logical backup of the Library Management System database was successfully created using `mysqldump`. The database was dropped, restored from the backup file, and post-recovery integrity was confirmed via join-based validation queries.

5. Conclusion

This practical emphasized the critical importance of backup and recovery as fundamental components of Database Administration. The `mysqldump` utility provides a reliable, portable mechanism for logical backups, and systematic post-restore validation ensures the database is production-ready after recovery.

Mini Project: Guidelines and Topic Selection

Note: This is a compulsory group project to be completed by 4-6 students. Database connectivity with a frontend and backend is mandatory.

Project Description

The Mini Project integrates all concepts covered in the DBMS practical course — relational design, SQL programming, PL/SQL routines, transaction management, and optionally NoSQL. The project simulates real-world applications by requiring a connected database backend and a working frontend interface.

Core Requirements

Requirement	Description
Schema Design	Normalized to at least 3rd Normal Form (3NF)
SQL Queries	Must include JOINS, aggregations, filters, and sorting
PL/SQL	At least one Stored Procedure, Function, Trigger, or Cursor
Transactions	Demonstrate ACID compliance with COMMIT/ROLLBACK
Connectivity	Backend (Python/Node.js/PHP) connected to a frontend interface
Documentation	ER Diagram, schema description, and query explanations

Instructions for Students

1. Select a topic from the approved list below or propose a unique idea to your instructor.
2. Form a group of exactly 4 to 6 students.
3. Design your schema — normalize tables, define relationships, and create an ER diagram.
4. Implement SQL scripts — schema, seed data, stored procedures, triggers, and analytical queries.
5. Build a frontend — a simple web or desktop interface for CRUD and reporting.
6. Connect the database — use a backend framework or direct JDBC/ODBC connection.
7. Document everything — write a brief report with the ER diagram, entity description, and sample outputs.

Approved Mini Project Topics (Extended — Batch 2026)

No.	Topic	Objective	Key Features
1	Library Management System	Manage books, users, loans, and overdue notifications	Book search with filters, loan and return history, overdue alerts
2	Online Retail Store Database	Ecommerce platform for products, customers, and inventory	Order tracking, real-time inventory updates, customer history
3	Student Attendance & Grade Management	Track attendance and academic performance for courses	Grade calculation, attendance reporting, performance analytics
4	Hospital Patient Management System	Manage patients, doctors, and appointments	Patient registration, medical history, doctor scheduling, billing
5	Movie Ticket Booking System	Seat selection and booking for movie shows	Show listings, seat availability, ticket cancellation, feedback
6	Food Delivery System	Coordinate restaurants, orders, and delivery	Menu management, order placement, delivery status tracking
7	Real Estate Property Management	Handle property listings, agents, and inquiries	Property filtering, customer inquiries, agent performance metrics
8	Social Media Analysis (NoSQL)	Analyze posts and hashtags using MongoDB	Trending hashtag identification, sentiment analysis, engagement data
9	Online Voting System	Secure election and vote management	Voter verification, secure tallying, audit trail, result announcement
10	Event Management System	Coordinate events, participants, and schedules	Attendee registration, event scheduling, notifications, feedback
11	Airline Reservation System	Manage flights, passengers, and seat reservations end-to-end	Flight search and filtering, seat selection, booking history, cancellation with fare rules, crew assignment
12	University Course Registration Portal	Allow students to register for courses with capacity and prerequisite enforcement	Prerequisite validation trigger, waitlist management, timetable conflict detection, GPA-based eligibility check, course analytics
13	Smart City Complaint Management	Capture and resolve civic complaints raised by citizens across departments	Category-based routing, priority scoring via stored function, SLA tracking, escalation trigger, resolution audit log
14	Banking Transaction Ledger	Record and audit financial transactions across customer accounts with fraud detection	Double-entry ledger, account balance trigger, fraud alert stored procedure, transaction rollback simulation, monthly statement generation

★ Topics 11–14 are newly added for Batch 2026. Groups selecting these topics are encouraged to incorporate triggers, stored procedures, and window functions for a higher evaluation score.

Reference Implementation — Universe Database System

Tip: This section presents a topper-level mini project reference. Study the design approach, relationship complexity, and query depth to understand what a high-quality submission looks like.

Project Title: Space Exploration Universe Database System 1. Aim

To design and implement a fully normalized relational database for a Space Exploration Universe System, modeling hierarchical astronomical entities — Galaxies, Stars, Planets, and Moons — along with Space Missions tracked through a many-to-many relationship. The project demonstrates advanced SQL including window functions, multi-level joins, ACID-compliant transactions with savepoints, stored procedures, and strategic indexing for query optimization.

2. Theory

a) Concept Introduction

The Universe Database System integrates all core DBMS concepts — relational schema design, normalization, PL/SQL programming, transaction control, and query optimization — into a single cohesive application. It models real-world hierarchical data across astronomical entities and demonstrates how complex, multi-level relationships are handled in a production-grade relational database.

b) Entities and Relationships

The system comprises six entities connected through primary and foreign key relationships:

- **Galaxy:** Root entity. Stores galaxy name, type, age, and spherical classification.
- **Star:** Child of Galaxy. Stores stellar properties — temperature, mass, and visibility.
- **Planet:** Child of Star. Stores planet type, habitability flag, and distance from star.
- **Moon:** Child of Planet. Stores moon age and shape classification.
- **Space_Mission:** Independent entity tracking missions by agency and type.
- **Mission_Planet (Junction Table):** Resolves the many-to-many relationship between missions and planets visited.

Entity	Primary Key	Foreign Key(s)
Galaxy	galaxy_id	—
Star	star_id	galaxy_id → Galaxy
Planet	planet_id	star_id → Star
Moon	moon_id	planet_id → Planet
Space_Mission	mission_id	—
Mission_Planet	(mission_id, planet_id)	Both → parent tables

c) Key Concepts Demonstrated

- One-to-Many Relationships: Galaxy → Star → Planet → Moon (hierarchical chain)
- Many-to-Many Relationship: Space_Mission ↔ Planet resolved via the Mission_Planet junction table
- Composite Primary Key: Mission_Planet uses (mission_id, planet_id) as a composite PK
- Foreign Key Cascading: ON DELETE CASCADE propagates deletions down the hierarchy

- Indexes: Applied on foreign keys and analytical filter columns for optimized query execution
- Transactions with SAVEPOINT: ACID compliance demonstrated with atomic inserts and rollback capability
- Window Functions: RANK(), DENSE_RANK(), ROW_NUMBER() for analytical ranking
- Multi-level JOIN Aggregation: Four-table joins for habitability distribution analysis
- Third Normal Form (3NF): No transitive dependencies; all non-key attributes depend only on the primary key
- Stored Procedure: Encapsulates mission insertion logic with error handling
- EXPLAIN: Query execution plan analysis for performance validation

d) SQL Syntax and Sample Queries Schema Creation (Sample):

```
CREATE TABLE galaxy (
galaxy_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL, galaxy_type
VARCHAR(50), age_in_millions_years INT, is_spherical
BOOLEAN DEFAULT FALSE );

CREATE TABLE star (
star_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL, galaxy_id INT NOT
NULL, temperature INT, mass DECIMAL(10,2),
is_visible BOOLEAN DEFAULT TRUE, FOREIGN KEY (galaxy_id) REFERENCES
galaxy(galaxy_id) ON DELETE CASCADE
);

CREATE TABLE mission_planet ( mission_id INT NOT NULL, planet_id INT NOT
NULL, visit_date DATE, PRIMARY KEY (mission_id, planet_id), FOREIGN KEY
(mission_id) REFERENCES space_mission(mission_id), FOREIGN KEY (planet_id)
REFERENCES planet(planet_id)
);
```


Habitability Distribution Across Galaxies:

```
SELECT g.name AS galaxy_name,
COUNT(p.planet_id) AS total_planets,
SUM(p.has_life) AS habitable_planets, ROUND((SUM(p.has_life) / COUNT(p.planet_id)) * 100, 2) AS
habitability_pct
FROM galaxy g
JOIN star s ON g.galaxy_id = s.galaxy_id JOIN planet p ON
s.star_id = p.star_id
GROUP BY g.galaxy_id, g.name
ORDER BY habitability_pct DESC;
```

Star Temperature Ranking with Window Functions:

```
SELECT s.name AS star_name, s.temperature, g.name AS galaxy_name,
RANK() OVER (ORDER BY s.temperature DESC) AS temp_rank,
DENSE_RANK() OVER (ORDER BY s.temperature DESC) AS dense_rank,
ROW_NUMBER() OVER (ORDER BY s.temperature DESC) AS row_num
FROM star s
JOIN galaxy g ON s.galaxy_id = g.galaxy_id;
```

Transaction with SAVEPOINT (ACID Demonstration):

```
START TRANSACTION; INSERT INTO space_mission (name, launch_year, agency,
mission_type) VALUES ('Deep Survey', 2026, 'ISRO', 'Research');
SAVEPOINT mission_saved; INSERT INTO mission_planet (mission_id, planet_id,
visit_date)
VALUES (LAST_INSERT_ID(), 3, '2026-07-10'); -- On error:
ROLLBACK TO SAVEPOINT mission_saved;
COMMIT;
```

Stored Procedure with Error Handling:

```
DELIMITER //
CREATE PROCEDURE sp_add_mission(
IN p_name VARCHAR(100), IN p_year INT, IN p_agency
VARCHAR(100), IN p_type VARCHAR(50))
BEGIN DECLARE EXIT HANDLER FOR
SQLEXCEPTION
BEGIN
ROLLBACK;
RESIGNAL;
END;
START TRANSACTION; INSERT INTO space_mission (name, launch_year, agency,
mission_type) VALUES (p_name, p_year, p_agency, p_type);
COMMIT;
END //
DELIMITER ;
```

e) Normalization Justification

- **1NF:** All attributes are atomic; no repeating groups exist in any table.
- **2NF:** All non-key attributes depend fully on the primary key. In mission_planet, visit_date depends on the full composite key (mission_id, planet_id).
- **3NF:** No transitive dependencies. Galaxy details are not stored in the Star table — they are referenced via foreign key only.

f) Advantages / Applications

- Demonstrates full-stack DBMS capability in a single integrated system.
- Window functions enable analytical ranking without application-layer processing.
- Cascade deletes enforce referential integrity automatically across the hierarchy.

- SAVEPOINTS allow partial rollback within a transaction, critical for complex multi-step inserts.
- Strategic indexing on FK columns ensures join operations scale with data volume.
- Applicable to astronomy research platforms, mission planning tools, and data science pipelines.

3. ER Diagram

The following entity-relationship diagram represents the Space Exploration Universe Database System, showing the hierarchical chain Galaxy → Star → Planet → Moon and the many-to-many relationship between Space_Mission and Planet through the Mission_Planet junction table.

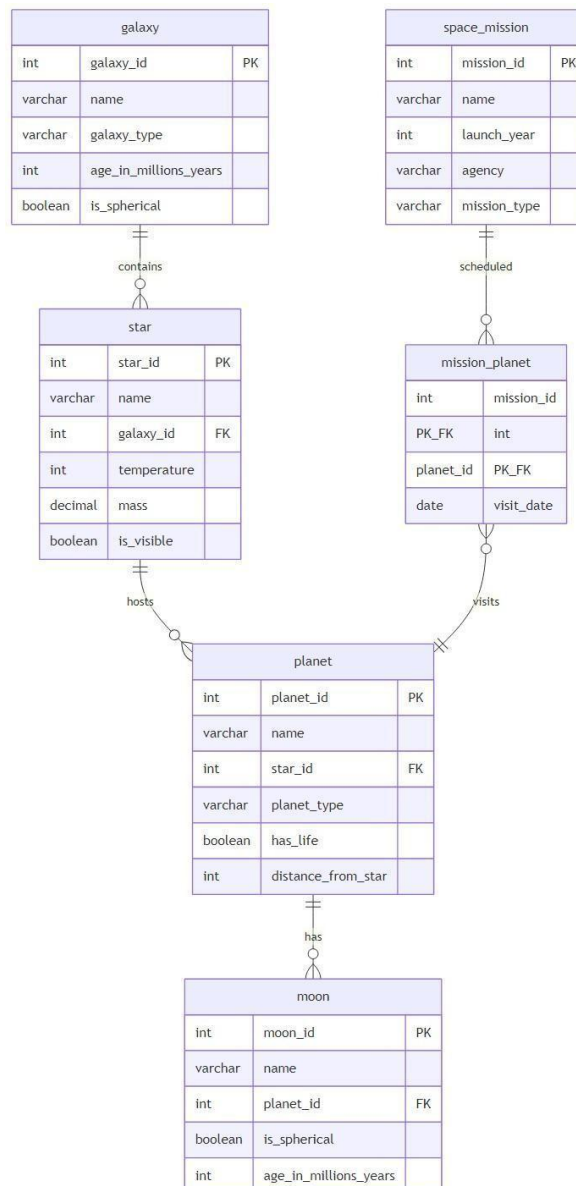


Figure: ER Diagram representing the Space Exploration Universe Database System

4. Result

The Universe Database schema was successfully created with six normalized tables — galaxy, star, planet, moon, space_mission, and mission_planet — each enforcing referential integrity through primary and foreign key constraints with ON DELETE CASCADE. All analytical queries including habitability distribution across galaxies, star temperature ranking using window functions (RANK, DENSE_RANK, ROW_NUMBER), and mission frequency per planet were executed successfully and returned correct, ordered result sets. The ACID-compliant transaction with SAVEPOINT was demonstrated by inserting a mission, rolling back to the savepoint on simulated failure, and committing successfully. The stored procedure sp_add_mission

encapsulated transactional logic with error handling. Query performance was validated using EXPLAIN, confirming that indexes on FK columns reduced full-table scans to efficient key lookups.

5. Conclusion

This mini project demonstrated the complete integration of DBMS concepts at an advanced level. The hierarchical schema design reinforced one-to-many and many-to-many relationship modeling, while the junction table pattern cleanly resolved the Mission-Planet association. The use of window functions highlighted the analytical power available natively in SQL without requiring post-processing in application code. SAVEPOINT-based transactions illustrated fine-grained control over atomicity, and the stored procedure showcased how database-level programming improves maintainability and security.

6. Learning Outcomes

- Design normalized schemas with complex multi-level hierarchical relationships.
- Implement advanced SQL including window functions and multi-join aggregations.
- Apply transactional control with SAVEPOINTS for data integrity across related tables.
- Use stored procedures to encapsulate business logic at the database tier.
- Optimize database performance through strategic indexing and EXPLAIN-based analysis.
- Demonstrate ACID compliance using MySQL's transaction control commands.